Space Debris Tracking and Avoidance using Data Structures and Algorithms

Vedant Pathak, Dhairya Shah, Kanishak Thakran

SVKM's NMIMS MPSTME, Mumbai, India

Source code and related work:
https://github.com/withvedantpathak/Space_Debris_Tracking_And_Avoidance_using_DataStructuresAndAlgorithms

Abstract: Checking every pair of tracked objects for a close approach grows with the square of the catalogue size. We build a complete conjunction-screening system in hand-written C99, with no external libraries in the core, and compare four ways to find candidate pairs: brute force, an altitude filter, a hashed uniform grid and a k-d tree. A proven distance bound lets the k-d tree and the grid skip pairs safely, so their event files are byte-identical to brute force in every run we compared (88 of 88 synthetic runs up to 10,000 objects, plus 20,000 synthetic and the real catalogue). At 20,000 objects they run in about 10 s instead of 98 s, with about 15,000 times fewer pair checks. On the real CelesTrak catalogue (19,440 objects, 18,191 in a 1-hour window) a 10 km screening distance gives 12,504 events on 11,914 pairs. These pairs form one giant cluster of 7,197 objects, while a 5 km distance keeps the largest cluster at 24. A greedy plan to clear all conjunctions is within 1.4% of the optimum that an independent integer-programming check confirms. The code is covered by 45,678 automated checks, 99.9% line coverage and a mutation check that kills 36 of 39 mutants. Events come from SGP4 element sets without covariance, so we report geometry and not collision probability.

Keywords: conjunction screening, space debris, k-d tree, spatial hashing, union-find, vertex cover, SGP4

I. INTRODUCTION

A conjunction is a predicted close approach between two orbiting objects. Operators screen the whole catalogue for them, because a collision with debris can destroy a satellite and create more debris. Screening is expensive because the obvious method compares every pair of objects at every time step. For the real CelesTrak catalogue used here (18,191 objects in low Earth orbit), one hour at a 10 s step already costs about 6 x 10^{10} pair checks with brute force. The cost grows with the square of the catalogue size.

This project builds a complete screening system and studies which data structures make it fast without losing any conjunction. Everything in the core is written from scratch in C99: dynamic array, queue, stack, binary heap, hash table, AVL tree, k-d tree, hashed uniform grid, union-find, graph search and a vertex-cover search. We use no external library. The pipeline is: load element sets, propagate every object to a common time grid, find candidate pairs with a spatial index, refine each candidate to its time of closest approach (TCA), build a conjunction graph, rank and cluster the events, and plan which objects to move.

Contributions.

1. A working end-to-end pipeline on the real catalogue and on seeded synthetic catalogues with planted pairs whose answers are known exactly.
2. A head-to-head comparison of brute force, an altitude filter, a hashed grid and a k-d tree, with a distance bound that makes the last two exact. Their outputs equal brute force byte for byte wherever we compared them.
3. A graph layer that turns events into clusters, a Top-K list and a mitigation plan, comparing a greedy heuristic with an exact search that an independent integer-programming solver confirms.
4. A verification layer (property tests, fault injection, sanitizers, coverage, mutation check) and a rule that every number in this report is read from a file in results/ produced by a script in scripts/.
5. Two honest findings: pure linear interpolation of closest approach invents events for slow, co-moving pairs (and a hybrid fixes it), and an age-aware screening distance changes the clusters strongly, with a measured typical error growth of about 2.5 km per day but a heavy tail that makes a high-percentile margin unusable.

Section II reviews related work, **Section III** fixes the scope and data, **Section IV** gives the design, **Section V** the methods, **Section VI** the verification, **Section VII** the experiments, and **Sections VIII and IX** the limitations and conclusions.

II. RELATED WORK

Deterministic conjunction tracking. Gómez, Gratl, Bösing and Izzo [1] track conjunctions in a 20-year simulation of 16,024 LEO objects. They find candidate pairs with the particle containers of the AutoPas library, use a fixed 10 s step, call two objects in conjunction when the closest-approach distance is at most kappa times the sum of their radii, and locate the closest approach inside a step by linear interpolation of positions and velocities. They name higher-order interpolation as future work. Their paper reports no comparison between spatial data structures and no scaling study over the object count. We follow their LEO scope (175 to 2,000 km), their 10 s step and their idea of a spatial structure that prunes pairs. We differ in the threshold (a fixed distance in km, because our element files carry no object radius), in the propagator (hand-written SGP4 over windows of hours to days, not a 20-year integrator), and in the closest-approach step (bisection on the exact distance derivative, with linear interpolation kept as a variant that we measure).

Tree-based pre-selection. Mercurio and Singla [2] compare all pairs in $O(N^2)$ and propose a k-d tree to pre-select pairs for detailed analysis, which they state is about O(N log N). Their emphasis is uncertainty: a Hellinger-distance nearest-neighbour search and Gaussian mixtures. We have read only the abstract of [2]. We therefore cite it

for the k-d tree idea and its complexity claim, and we do not reproduce or compare against its probabilistic part. Our system uses deterministic distances only.

Propagation. For real catalogue data we propagate with SGP4 [3, 4], written by hand in C and checked against the reference implementation (**Section VI**). Synthetic catalogues use a simpler Kepler, J2 and drag model, which is documented as such.

Element-set age. Predictions from public element sets grow worse with the age of the set. Levit and Marshall [5] report, in their abstract, about 100 m per day for an improved method, ten times better than element sets propagated with SGP4. That implies an order of magnitude of 1 km per day for plain SGP4, which is the only literature anchor we use for the age-aware distance in **Section VII**. We did not find a verified single figure and do not quote one.

Operational screening. CelesTrak SOCRATES [6] screens active payloads against the public catalogue and lists conjunctions within 5 km at the time of closest approach over the next seven days. This supports a screening distance of a few kilometres. Our default is 10 km and is configurable.

III. SCOPE, DATA AND ASSUMPTIONS

Scope. The system screens low-Earth-orbit objects only: perigee altitude of at least 175 km and apogee altitude of at most 2,000 km above a spherical Earth (R = 6378.137 km), following [1]. Deep-space and high-eccentricity objects need SDP4, which we did not write. Manoeuvres, collision probability and covariance are out of scope. The claim of this work is about screening speed and recall, not about the long-term evolution of debris.

Real data. The real catalogue is a CelesTrak snapshot taken on 2026-10-20: the active-satellite file plus three debris files (COSMOS 2251, FENGYUN 1C, IRIDIUM 33), 19,440 unique NORAD ids in OMM CSV form. After the LEO scope and an epoch filter (epochs within 3 days of the median epoch), 18,191 objects remain. Real-data runs use a 1 h window starting at the median epoch, a 10 s step and a 10 km threshold unless stated otherwise. The snapshot is kept in *data/raw/*; it is not redistributed with this report.

Synthetic data. A seeded generator (gen <n> <seed>) draws catalogues of 1,000 to 50,000 objects whose orbit-class mix is taken from histograms of the real snapshot, and can plant close pairs whose miss distance and time are known exactly. Planted pairs give an answer key that does not depend on any other code. **Table I** compares the two catalogues; the generator reproduces the LEO share well, but not the inclination distribution, so the synthetic timings are a scaling study, not a prediction for the real catalogue.

TABLE I
REAL AND SYNTHETIC CATALOGUES (RESULTS/DATASET_STATS.CSV).

Catalogue	Objects	LEO share	GEO share	Median LEO altitude (km)	Median LEO inclination (deg)
Real (CelesTrak, 2026-10-20)	19,440	95.8 %	3.0 %	483	53.2
Synthetic (20,000 objects, seed 1)	20,000	95.8 %	2.8 %	477	59.3

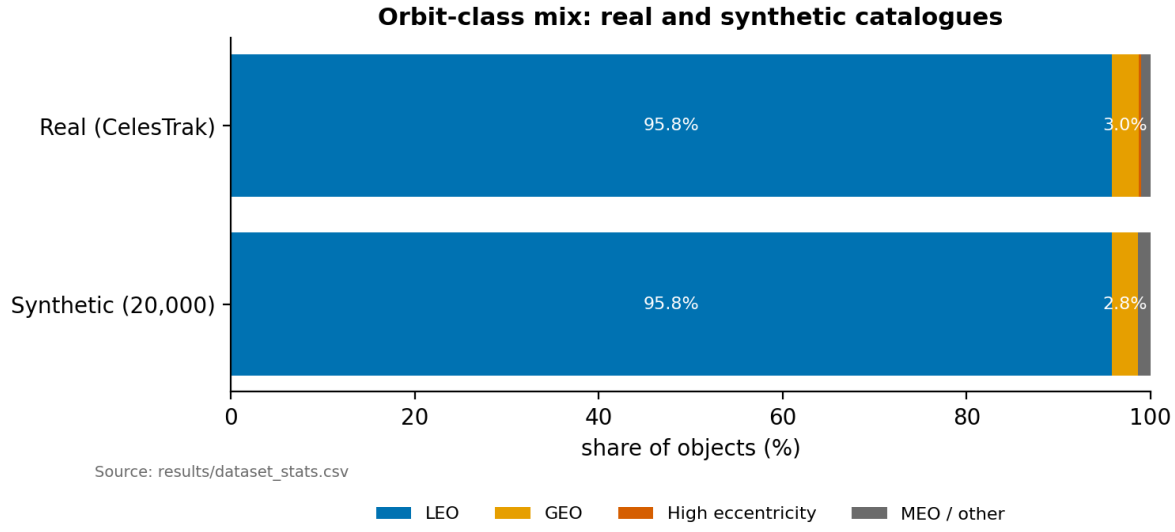


Fig. 1. Catalogue composition, real versus synthetic.

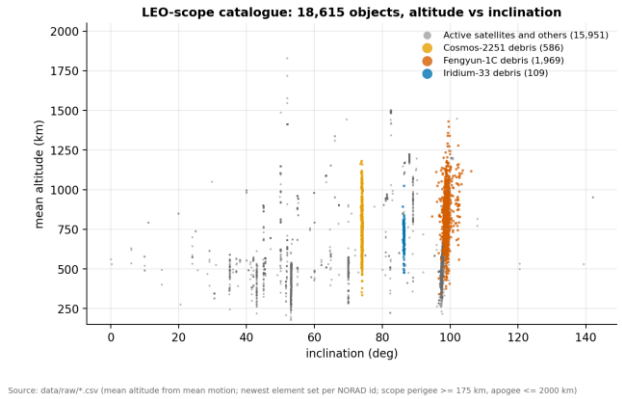


Fig. 2. Altitude and inclination of the real catalogue: the dense LEO shell and the debris bands.

Assumptions. (1) Objects are points; the distance threshold is a fixed number of km (default 10 km, configurable) because the element files carry no radius. (2) Events are SGP4 against SGP4, so they measure the screening algorithm, not agreement with reality (**Section VIII**). (3) Positions are in an inertial (TEME-like) frame without Earth rotation, which is enough for relative geometry. (4) Timings are single-thread CPU seconds measured with clock() on a virtual machine with two logical CPUs; absolute times are indicative and ratios are the result.

IV. SYSTEM DESIGN

The system is a C99 library plus a command-line program (*dlscms*). Fig. 3 shows the pipeline and the data structure each stage rests on. Each stage reads and writes plain CSV, so any stage can be run and checked on its own.

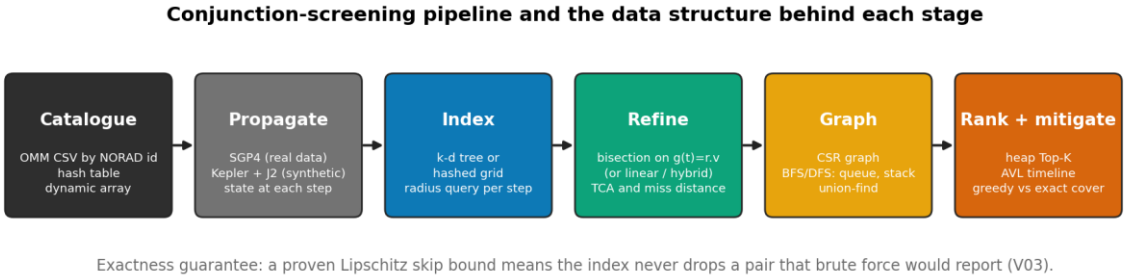


Fig. 3. Pipeline from catalogue to mitigation plan, with the data structure used at each stage.

1. **Catalogue.** Element sets are parsed into a dynamic array of objects. A hash table keyed by NORAD id gives constant-time lookup and rejects duplicates. The parser validates ranges (for example eccentricity below 1) and counts every rejected row.
2. **Propagation.** Each object is propagated to a common time grid with the hand-written near-earth SGP4 (real data) or the Kepler, J2 and drag model (synthetic data).
3. **Spatial index.** At each time step the positions go into a k-d tree or a hashed uniform grid. A radius query returns the candidate neighbours of an object.
4. **Closest-approach refinement.** For each candidate pair, the time of closest approach and the miss distance inside the step are found (**Section V-C**).
5. **Conjunction graph.** Objects are nodes and events are edges, stored in compressed sparse row (CSR) form. Breadth-first search (a queue), iterative depth-first search (a stack) and union-find give the clusters, and they must agree.
6. **Ranking and mitigation.** A bounded heap gives the Top-K closest encounters. An AVL tree indexed by TCA gives the timeline. A greedy heuristic and an exact branch-and-bound search choose which objects to move so that no event remains.

Table II maps each data structure to its use and cost. The costs are the standard ones for the structures as implemented, not measurements; **Section VII** measures what they add up to.

TABLE II
DATA-STRUCTURE MAP (SOURCE: DOCS/DSA_COVERAGE.MD).

Structure	Used for	Cost
Dynamic array	Catalogue, event lists, CSR arrays	Amortised O(1) append
Hash table (chaining)	NORAD id lookup; grid cells	O(1) average per operation
k-d tree	Radius query over positions at a time step	O(n log n) build with quickselect; radius query roughly O(log n + k) on spread-out points
Hashed uniform grid	Same query as the k-d tree	O(n) build; O(1) cell lookup on average
Binary heap	Bounded Top-K; greedy cover with lazy deletion	O(log n) per push or pop
AVL tree	Conjunctions ordered by TCA	O(log n) insert and range query
Queue	Breadth-first search, hop distance	O(1) per operation; search O(V + E)
Stack	Iterative depth-first search; undo of the exact cover search	O(1) per operation; search O(V + E)
Union-find	Cross-check of the components	Near-constant amortised per operation
Branch and bound	Exact minimum vertex cover	Exponential in the worst case; reductions and a matching lower bound prune it

Shortest-path and dynamic-programming techniques are not used: hop distances use unweighted BFS, and the cover problem is solved by search, not by a recurrence.

V. METHODS

A. Propagation

Synthetic catalogues use a Kepler model with secular J2 and a drag term. Against SGP4 on the real LEO catalogue, its median position error is 12.0 km at the epoch and 8.4 km six hours later with J2 and drag, while plain two-body error grows to 83.7 km after six hours (Fig. 4, *results/propagation_error.csv*). That error is too large to screen real data, so real-data runs use SGP4. Our near-earth SGP4 is hand-written and was compared with the Python

sgp4 package on all 19,440 real objects at 0 h, 1 h, 1 d, 3 d and 7 d: the largest position difference is 0.86 mm and the largest velocity difference 0.9 um/s, which is the print precision of the program output (Fig. 5, *results/sgp4_validation.csv*). Our code refuses exactly the 805 deep-space objects (period of 225 min or more) and never refuses an object the reference accepts.

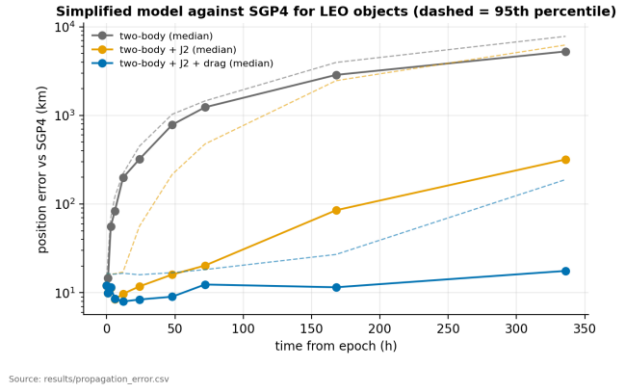


Fig. 4. Position error of the Kepler-family models against SGP4, by hours after epoch.

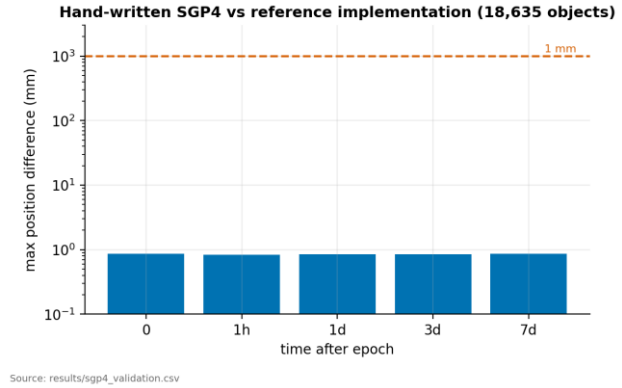


Fig. 5. Hand-written SGP4 against the reference implementation.

B. Candidate pairs and the exact skip bound

The baseline compares every pair at every step of h = 10 s. A pair can be skipped at a step if no event is possible in that step. If d0 is the distance at the start of the step and L = vmax_i + vmax_j bounds how fast the distance can change, then the distance stays above d0 - L*h in the step. So a pair with d0 > threshold + L*h cannot have an event and is skipped (Fig. 6). Brute force uses this bound to skip the refinement; the k-d tree and the grid use it to skip the pair check altogether. Each object queries the index with radius threshold + (vmax_i + vmax_max)*h, so any pair that could have an event is returned. For this reason both indexes are exact by construction, up to the 1 % margin we add to vmax. We also check it directly: on every data set we compared, their event files are byte-identical to the brute-force file.

The k-d tree is rebuilt at every step (O(n log n) with quickselect for the median) and the grid rebuilt in O(n). Fig. 7 sketches both. The grid cell size is set to the largest query reach. The altitude filter is a third method: sort objects by perigee radius and skip pairs whose radial ranges are further apart than the threshold plus a pad. It is a heuristic, not a proven bound, because under SGP4 the radius can leave the mean-element range (an excursion of up to 26.7 km was measured over 10 minutes; the CLI pad is 50 km).

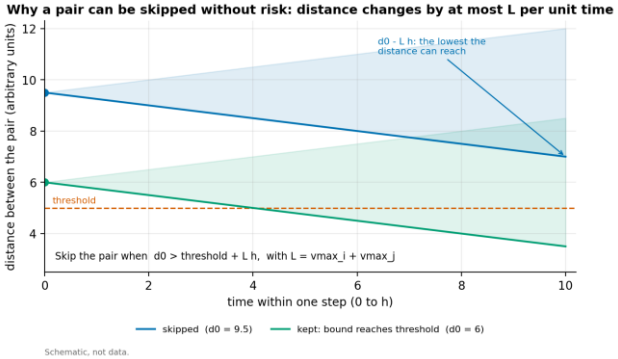


Fig. 6. Why a pair can be skipped without missing an event (Lipschitz bound).

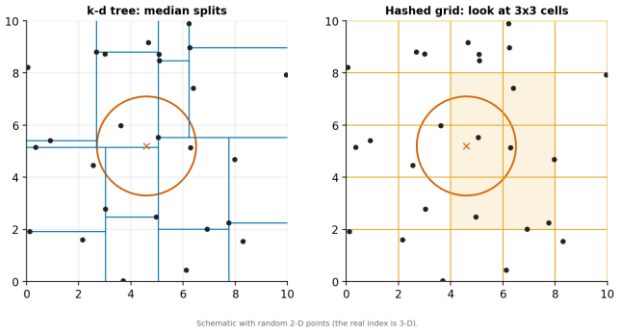


Fig. 7. The two spatial indexes: a k-d tree split and a hashed grid cell with the query radius.

C. Closest-approach refinement

For a candidate pair, the minimum of the distance inside a step is where g(t) = r . v changes sign, with r and v the relative position and velocity. We find the root by bisection on g(t) (Fig. 8) and take the miss distance and TCA there. Bisection uses the same propagator as the screen, so the result does not depend on the step (**Section VII**). Two cheaper variants are measured: *linear* refinement interpolates positions and velocities linearly across the step, as in [1], and *hybrid* uses linear interpolation for pairs with relative speed of at least 1 km/s and bisection below that. Bisection is the default.

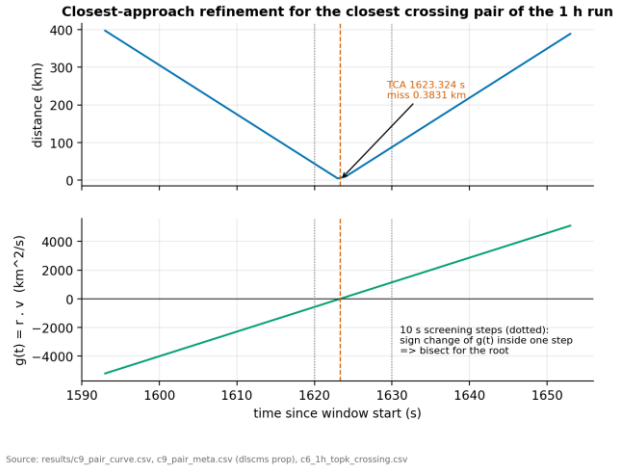


Fig. 8. Distance and r . v around a time of closest approach: the root of r . v is the minimum.

D. Agreement with an independent implementation

We compare the events of our SGP4 screen (1 h, 10 km, LEO scope) with the events of an independent Python implementation (*scripts/event_agreement.py*). All 12,501 reference events are found by our screen, and the median miss-distance difference is 0.00 km. Our screen reports

12,504 events. The three extra events have a TCA within 0.0006 s of a 10 s grid time, which the Python reference excludes by a guard. This is agreement between two SGP4 screens, not agreement with reality (**Section VIII**).

The Kepler-family model of **Section V-A** agrees much less well with SGP4 events. Screening at the same distance, it finds only 20 % of the 2,254 SGP4 events below 5 km and 55 % of the 12,504 below 10 km. It needs a screening distance of 25 km to find 99.4 % and 99.7 % of them respectively (Fig. 9, *results/event_agreement.csv*). This is why real-data results use SGP4 and why the Kepler-family model is used only for synthetic catalogues, where it is the ground truth.

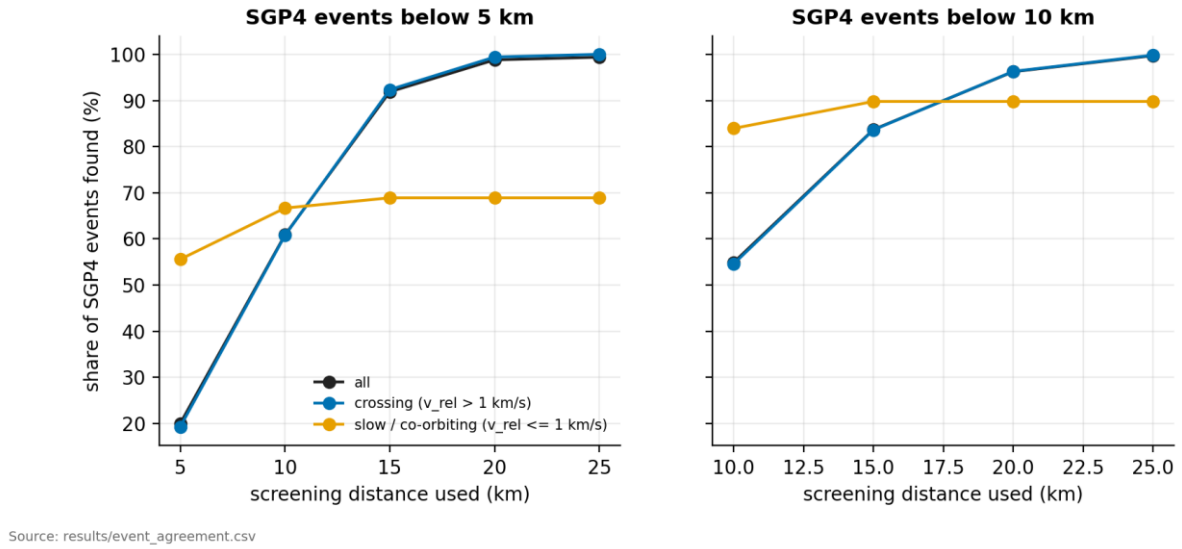


Fig. 9. Share of SGP4 events found by the Kepler-family model, against the screening distance it uses.

E. Conjunction graph, clusters and ranking

Events become a graph: a node for each object with at least one conjunction and an edge for each pair that has one, weighted with its closest miss distance (CSR storage, id-to-node mapping by the hash table). Components are found three ways, by breadth-first search with a queue, by iterative depth-first search with an explicit stack and by union-find, and the three must agree. Hop distance from the busiest object uses BFS. The Top-K closest encounters use a bounded binary heap in O(n log K), checked against a full sort, and a second list keeps crossing encounters only (relative speed above 1 km/s), since the closest encounters overall are mostly objects drifting together. An AVL tree keyed by TCA gives the number of events per 10-minute bin. Fig. 10 draws the largest cluster at 5 km; the one at 10 km has 7,197 objects and is too large to draw.

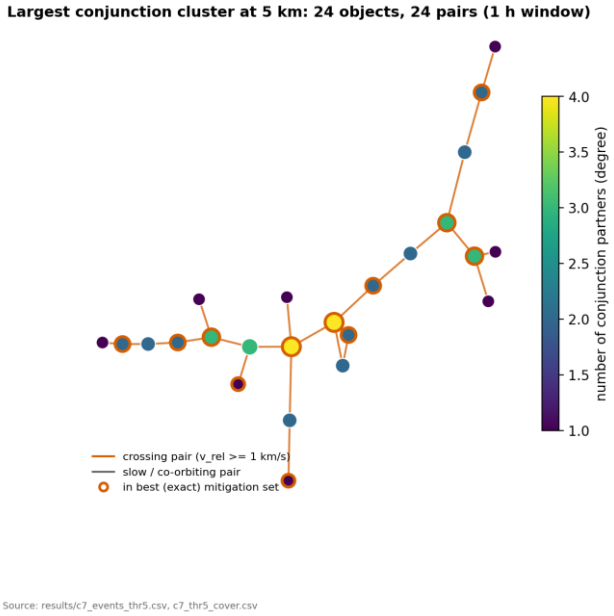


Fig. 10. Largest conjunction cluster at 5 km (24 objects, 24 pairs, 1 h window); ringed nodes form the exact mitigation set.

F. Mitigation as vertex cover

An object that manoeuvres is assumed to remove all its conjunctions. Choosing a smallest set of movable objects (active satellites; debris cannot manoeuvre) so that every event has a moved object is a minimum vertex cover. A pair with no movable object cannot be cleared and is reported as unresolvable. We implement a greedy heuristic (highest degree first, using a heap with lazy deletion, followed by a pass that drops redundant picks) and an exact search: reductions, independent groups, branch and bound with a matching lower bound, and an undo stack. An independent integer-programming solver (*scripts/verify_cover.py*) checks the optimum.

VI. VERIFICATION

The fast methods are only useful if they give the same answer as brute force, so verification is part of the result. **Table III** shows the main check: every run of the k-d tree and grid screens gave an event file identical to brute force.

TABLE III
FAST METHODS AGAINST BRUTE FORCE
(RESULTS/C7_SCALING.CSV, 88 SYNTHETIC RUNS; REAL RUN FROM RESULTS/BENCH_C5_SUMMARY.CSV AND THE EVENT FILES).

Data	Seeds	Runs compared	Identical to brute force	Events
Synthetic, N = 1,000	3	27	27 of 27	29 to 34 per seed
Synthetic, N = 2,000	3	27	27 of 27	about 100 on average
Synthetic, N = 5,000	3	27	27 of 27	about 606 on average
Synthetic, N = 10,000	1 to 3	7	7 of 7	2,288
Real catalogue (SGP4), 18,191 objects	1	1 per method	byte-identical event files	12,504

Beyond this, the code is checked in layers. Everything is reproducible with *scripts/test.sh* and the scripts named below.

- **Unit and example tests** for every module, and **property tests** for structural invariants (heap, k-d tree, grid, union-find, AVL), parser fuzzing, scope filters against naive predicates, 120 random screening configurations of kd, grid and the altitude filter against the brute-force baseline, and metamorphic relations (permuting the input, monotone threshold, window prefix).
- **Out-of-memory injection.** The k-th allocation fails, once and then from k on: 8 scenarios and 6,550 injected failures. Each must give an error or the unfailed result, with no crash and no leak.
- **Sanitizers.** Every test runs twice: plain -O2, and with AddressSanitizer (including leak detection), UBSan and float-divide-by-zero and float-cast-overflow checks, aborting on the first error.
- **Warnings and static analysis.** -std=c99 -Wall -Wextra -pedantic builds without warnings; gcc -fanalyzer reports none.
- **Coverage.** 99.9 % of lines (1,565 of 1,567) and 91.8 % of branches (1,623 of 1,768) across the library modules (*results/c8_coverage.csv*). The two unexecuted lines are defensive returns in the generator. Branch coverage is lower mainly because of error paths.
- **Mutation check.** 39 hand-written single-line mutants; 36 are killed (92 %). The three survivors are judged equivalent by inspection (equal-key order in the heap, the top-K tie rule, an event exactly at the window end), not proven so. The first run left six mutants alive; they exposed real gaps in the tests, which we then closed.

In total the test programs run 45,678 checks with no failure (Fig. 11, *results/c8_test_counts.csv*).

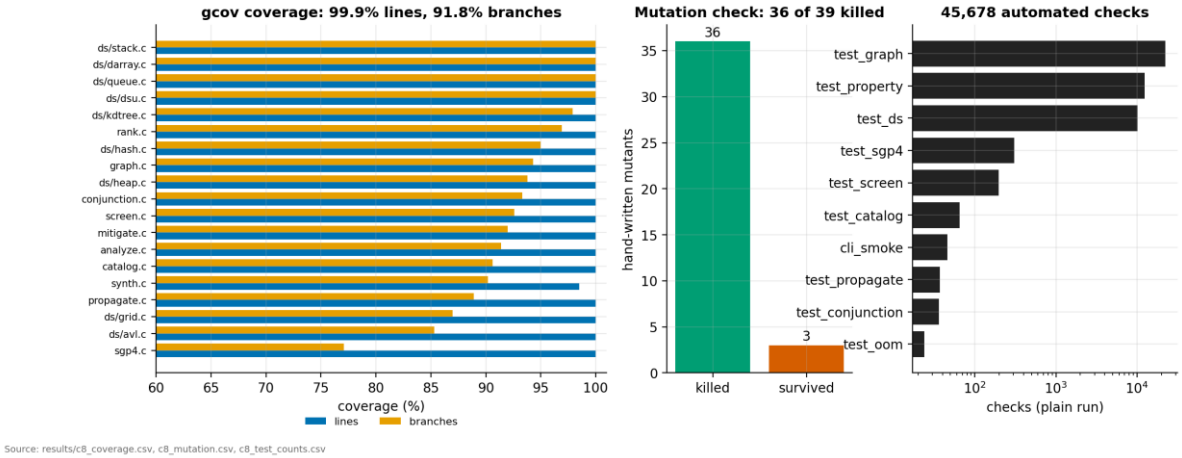


Fig. 11. Checks per test program, line and branch coverage per module, and mutation results.

These checks found real defects, all fixed: a NUL byte in a CSV row made the loader swallow the next row; step 0 or NaN looped forever; the grid returned an error for an empty catalogue; a failed initialisation left a hash table unusable and then crashed; the analysis overflowed a 32-bit time key for large TCA values; the command line returned success for unknown commands; and several out-of-memory paths left inconsistent state. The full list is in *docs/verification.md*.

What is not verified. Nothing has been run on Windows, where long is 32 bits (the .bat scripts exist but have never been executed). SGP4 accuracy against reality is not tested; events are SGP4 against SGP4. The out-of-memory injection fails only the project's own allocations and works on Linux with GNU ld only. **Section VIII** lists the rest.

VII. EXPERIMENTS AND RESULTS

All runs are single-threaded; times are CPU seconds from clock() on one virtual machine (*results/environment.txt*). Ratios matter more than absolute times. Every figure below is drawn from files in results/ by *scripts/plot_c9.py*.

A. Scaling with catalogue size

Table IV gives CPU time for the three exact methods on synthetic catalogues (1 h window, 10 s step, 10 km threshold) and on the real catalogue. All methods return the same events (**Table III**). The k-d tree and the grid examine about 15,000 times fewer pairs than brute force (14,900 to 15,800 across the synthetic sizes from 1,000 to 20,000; 11,452 on the real catalogue: 5,200,949 against 59,560,972,200 checks), but the time falls less, by 10 times at 20,000 objects and 6.6 to 6.9 times on the real catalogue (Figs. 12 and 13). After the index, the remaining cost is

propagation and the refinement of the pairs that survive the skip bound. The k-d tree and grid are within about 10 % of each other everywhere and neither wins consistently. Peak memory grows slowly and is not the bottleneck (Fig. 14).

TABLE IV
CPU TIME BY METHOD (RESULTS/C7_SCALING.CSV: MEAN OF 3 SEEDS AND 3 REPEATS FOR 1,000 TO 5,000, AND FOR 10,000 THE MEAN OF 3 K-D AND GRID RUNS AGAINST ONE BRUTE-FORCE RUN; RESULTS/BENCH_C5_SUMMARY.CSV FOR 20,000 AND THE REAL CATALOGUE; RESULTS/C7_SCALING_50K.CSV FOR 50,000, ONE RUN).

Objects	Brute force (s)	k-d tree (s)	Grid (s)	Speed-up, k-d / grid
1,000	0.30	0.15	0.19	2.0 / 1.6
2,000	1.16	0.36	0.40	3.3 / 2.9
5,000	6.82	1.18	1.14	5.8 / 6.0
10,000	26.6	3.13	2.87	8.5 / 9.3
20,000	98.2	9.79	9.20	10.0 / 10.7
50,000	not run	45.4	41.3	not available
Real, 18,191	102.3	15.6	14.8	6.6 / 6.9

At 50,000 objects we measured only the k-d tree and the grid. Both give the same 56,780 events. We did not run brute force there, because it would take hours on this machine; we state no speed-up at that size (**Section VIII**).

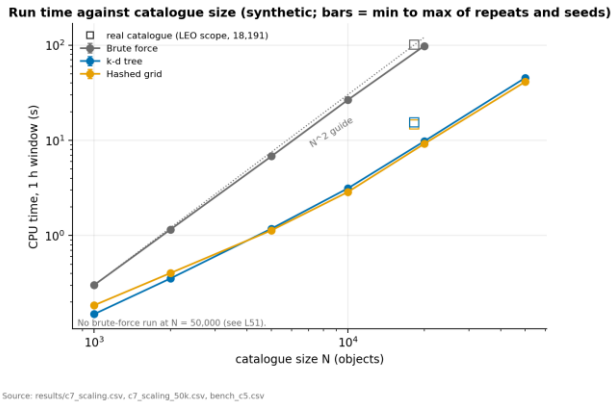


Fig. 12. CPU time against catalogue size, log-log.

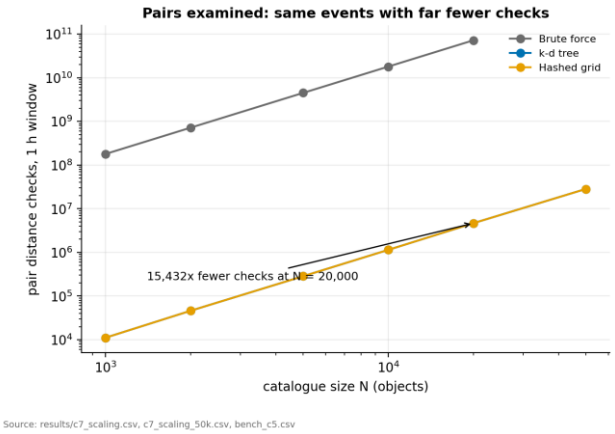


Fig. 13. Pair checks against catalogue size, log-log.

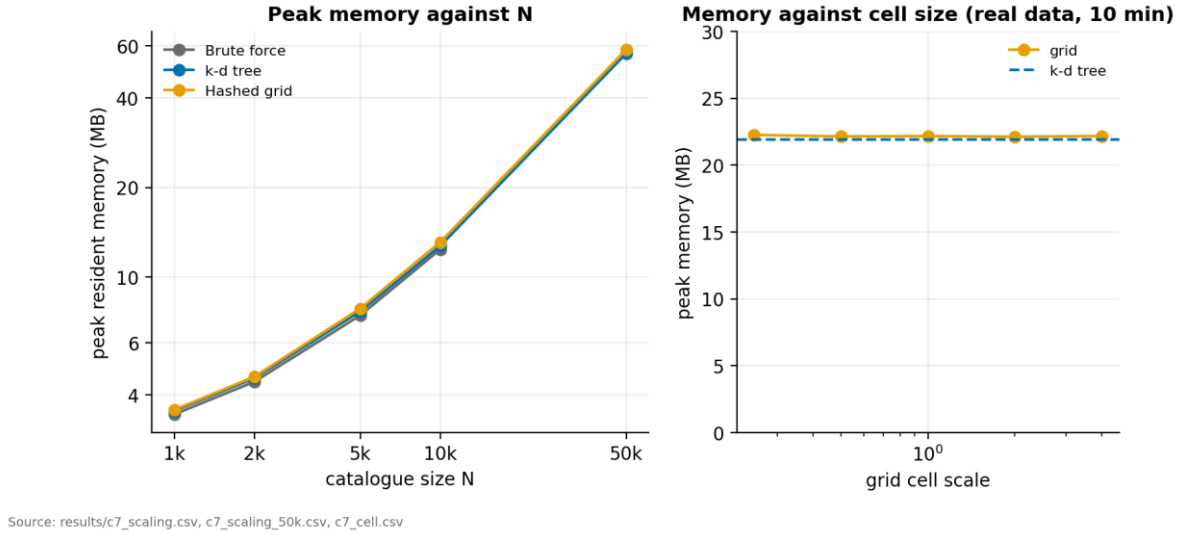


Fig. 14. Peak memory against catalogue size.

The micro-benchmark of the structures alone supports the same reading. For one all-pairs radius search within 50 km at a single instant, brute force takes 821 ms at 50,000 objects, the grid 75 ms and the k-d tree 14 ms, all with recall 1.0 (Fig. 15, *results/bench_c1.csv*). At 1,000 objects the grid is slower than brute force (1.2 ms against 0.34 ms), so the structures pay off only from a few thousand objects upward.

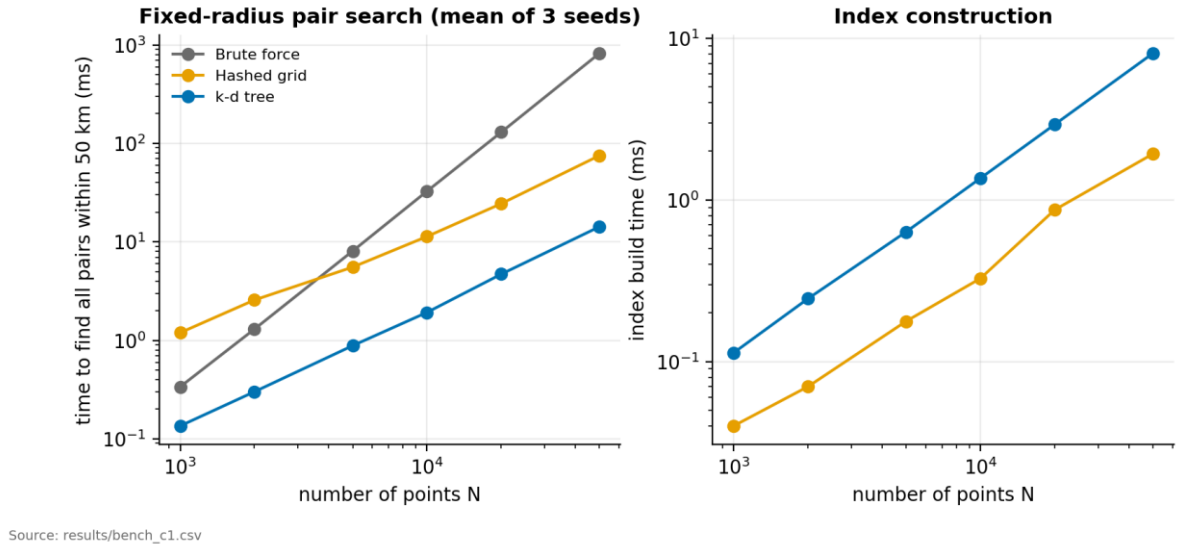


Fig. 15. Radius-search time of the hand-written structures against brute force.

B. Tuning and a negative result

Grid cell size. On a 10 min window at 5 km, every cell scale returns the same events and pair checks. Time is best at 1 times the query reach (2.18 s), four times slower at 0.25 (8.88 s), and 2.75 s at four times the reach. The k-d tree takes 2.29 s. Memory is about 22.5 MB for all settings (Fig. 16, *results/c7_cell.csv*).

Step size. With bisection, the 2,254 events at 5 km are identical for steps from 5 s to 60 s, with zero difference in miss distance and TCA. Only cost changes: candidates grow from 375,749 to 7,078,327 and CPU time from 13.1 s to 88.7 s, so the smallest step tested (5 s) is cheapest here (Fig. 17, *results/c7_step.csv*).

Altitude filter. Sorting by perigee radius is slower than brute force for LEO, 0.69 to 0.86 times its speed on the synthetic catalogues, because radial ranges overlap and the filter cuts the pair checks only about 2.5 times (Fig. 18, *results/bench_c5_summary.csv*). We report it as a negative result. It might help on a catalogue mixing LEO, MEO and GEO, which is out of scope.

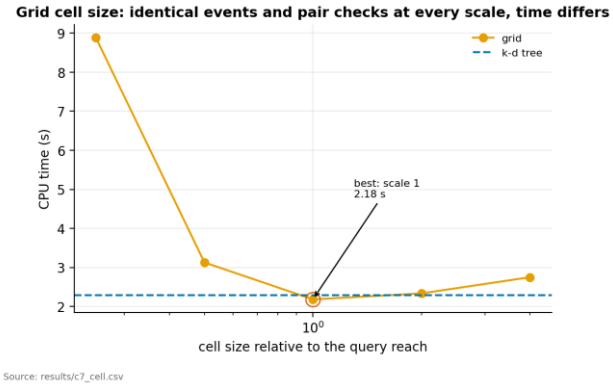


Fig. 16. Grid cell-size sensitivity.

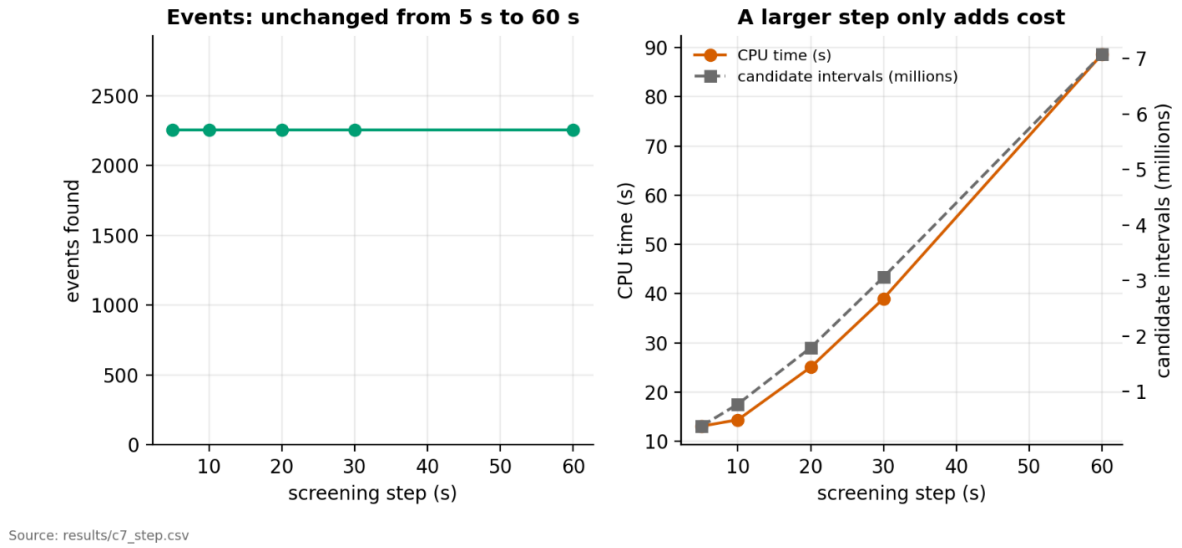
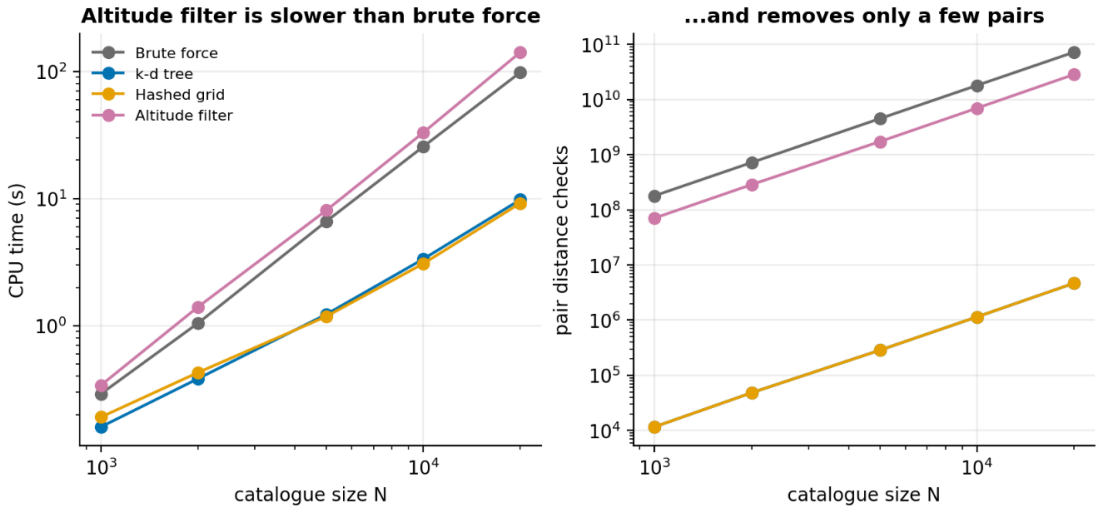


Fig. 17. Step-size study: events and cost against step.



Source: results/bench_c5.csv

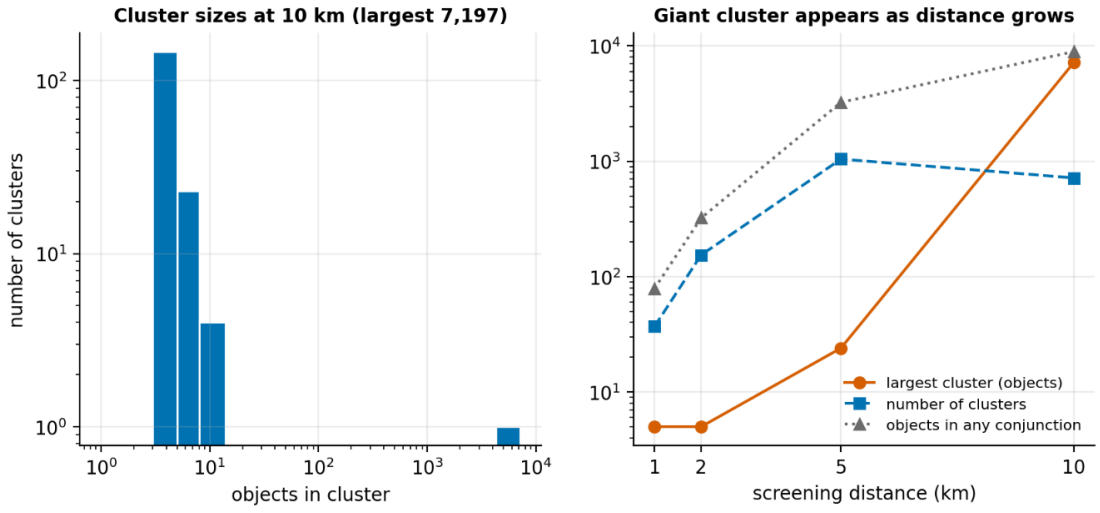
Fig. 18. Altitude filter against brute force.

C. Threshold and clusters

On the real catalogue (1 h, SGP4), the number of events rises steeply with the threshold: 48, 174, 2,254 and 12,504 at 1, 2, 5 and 10 km. The largest cluster in the conjunction graph holds 5, 5, 24 and 7,197 objects (**Table V**). Between 5 and 10 km the graph becomes one giant cluster of 7,197 objects together with hundreds of small ones (Fig. 19). At 10 km, 717 clusters exist, 542 of them single pairs. Events are spread evenly in time: at 5 km, 353 to 383 pairs have their TCA in each 10-minute bin (Fig. 20).

TABLE V
THRESHOLD SWEEP (RESULTS/C7_THRESHOLD.CSV).

Threshold (km)	Events	Objects in events	Clusters	Largest cluster	Greedy cover	Exact cover (= MILP optimum)
1	48	79	37	5	39	39
2	174	322	154	5	154	154
5	2,254	3,242	1,044	24	1,353	1,346
10	12,504	8,923	717	7,197	3,737	3,687



Source: results/c7_thr10_components.csv, c7_threshold.csv

Fig. 19. Cluster-size distribution and the largest cluster against threshold.

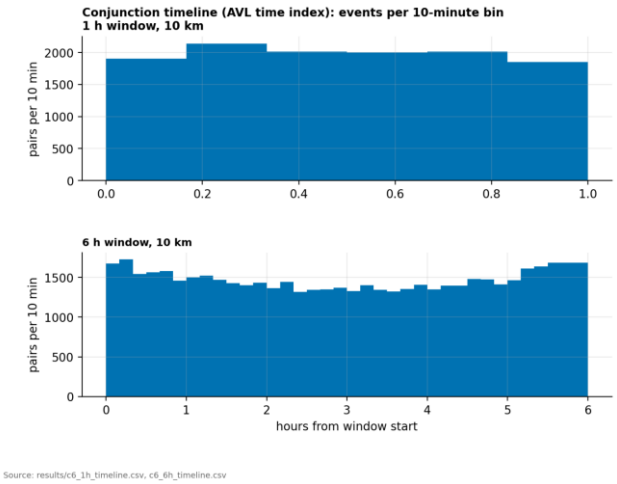


Fig. 20. Conjunction pairs per 10-minute bin.

The closest encounters overall are dominated by objects drifting together at near-zero relative speed, such as launch batches. We therefore also list the closest *crossing* encounters, with relative speed above 1 km/s (**Table VI**). The closest has a miss distance of 0.383 km at 13.1 km/s. Fig. 21 shows it among all objects at its TCA.

TABLE VI
CLOSEST CROSSING ENCOUNTERS, 1 H WINDOW AT 10 KM (RESULTS/C6_1H_TOPK_CROSSING.CSV). TCA IS SECONDS FROM THE START OF THE WINDOW.

Rank	NORAD ids	TCA (s)	Miss (km)	Relative speed (km/s)	Cluster size
1	52383, 55939	1,623.3	0.383	13.1	7,197
2	52744, 53275	3,447.2	0.434	14.3	7,197
3	51951, 55935	2,410.7	0.470	12.8	7,197
4	67089, 100143	3,002.9	0.506	13.8	4
5	48093, 65529	535.7	0.579	14.7	7,197

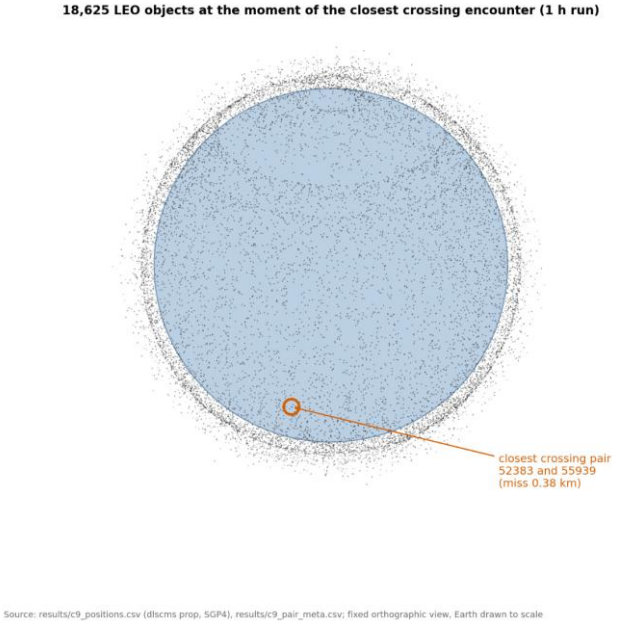


Fig. 21. All objects at the time of the closest crossing encounter, with the pair highlighted.

D. Mitigation

Table VII compares the greedy cover, the exact search and an independent integer-programming optimum on windows of 1, 3 and 6 h at 10 km. The exact search matches the independent optimum at 1 h and 6 h. At 3 h one cluster exceeded the node limit of 300,000 and the search stopped at 4,839 against the optimum 4,832, so there the optimum is not proven by our search. Greedy is 0.2 to 1.4 % above the optimum and is strictly worse than optimal in 1, 2 and 8 clusters (Fig. 22). Reductions are decisive: without them the giant cluster at 1 h is not solved within the node limit (174 of 175 non-trivial clusters finish), and with them the whole 1 h instance takes 0.65 ms (Fig. 23). The cover rests on the assumption that a manoeuvre removes all of an object's conjunctions, which is a modelling choice and not a flight-dynamics result (**Section VIII**).

TABLE VII
MITIGATION (RESULTS/C6_SUMMARY.CSV, RESULTS/C6_*H_VERIFY.CSV).

Window	Pairs	Objects	Clusters	Largest	Greedy	Exact search	MILP optimum	Lower bound
1 h	11,914	8,923	717	7,197	3,737	3,687	3,687	3,310
3 h	29,229	12,005	950	9,416	4,842	4,839 (cut off)	4,832	4,342
6 h	52,870	14,306	895	11,045	5,614	5,595	5,595	4,921

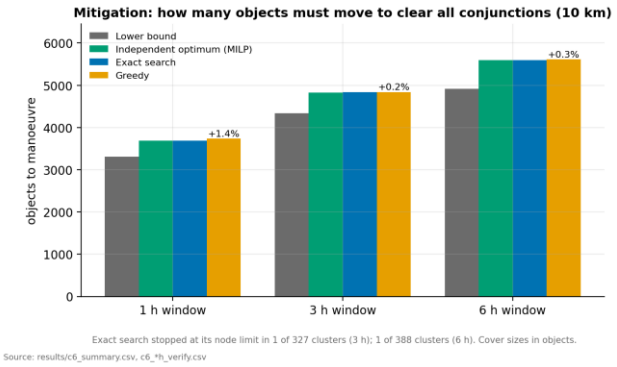


Fig. 22. Greedy, exact, lower bound and independent optimum.

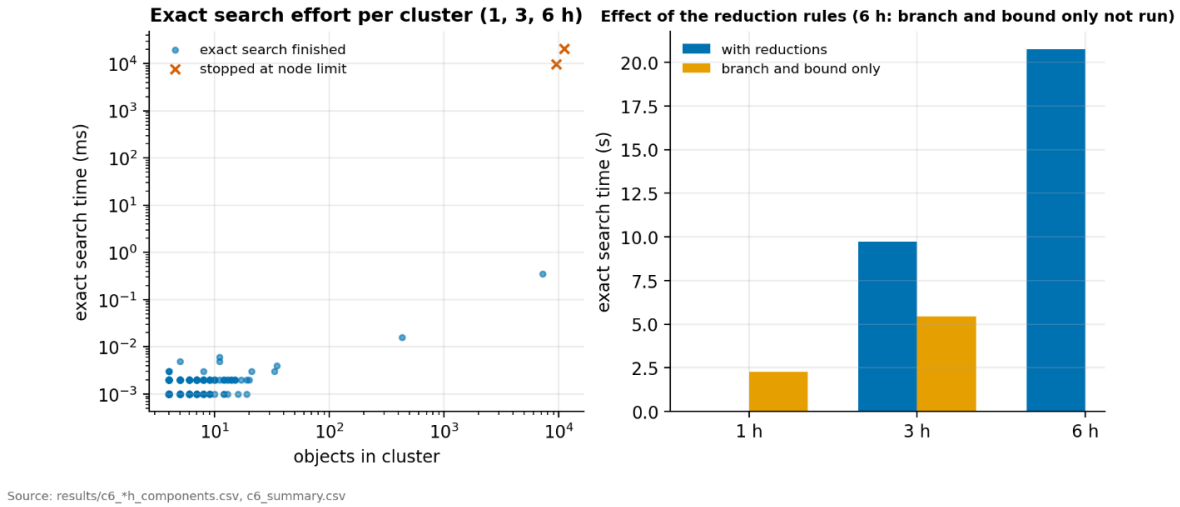


Fig. 23. Effort of the exact search, with and without reductions.

E. Linear interpolation against bisection

Pure linear interpolation of the closest approach, as in [1], is much cheaper but invents events. At a 10 s step it reports 2,450 events against 2,254 from bisection (recall 0.9956, precision 0.9159), and CPU time falls from 14.3 s to 5.7 s (*results/c7_interp.csv*). All 206 extra events are pairs with a relative speed below 0.5 km/s, that is, objects that move together and whose distance has no sharp minimum. For crossing encounters (relative speed above 1 km/s) linear interpolation matches bisection exactly at a 10 s step (2,209 of 2,209, no extra events). The *hybrid* variant, linear above 1 km/s and bisection below, removes the extra events and recovers all 2,254 events at 5 s and 10 s (**Table VIII**, Fig. 24). At 20 s and above it loses one to four events, because the linear estimate itself misses. The hybrid is not the default; the cost of its speed-up has not been measured here, so we leave bisection as the safe default.

TABLE VIII
REFINEMENT VARIANTS AGAINST BISECTION, WHICH GIVES 2,254 EVENTS (RESULTS/C8_L33_INTERP.CSV).

Step (s)	Linear: events	Linear: precision	Linear: recall	Hybrid: events	Hybrid: recall
5	2,446	0.920	0.998	2,254	1.000
10	2,450	0.916	0.996	2,254	1.000
20	2,444	0.919	0.996	2,253	1.000
30	2,442	0.917	0.993	2,253	1.000
60	2,437	0.916	0.991	2,250	0.998

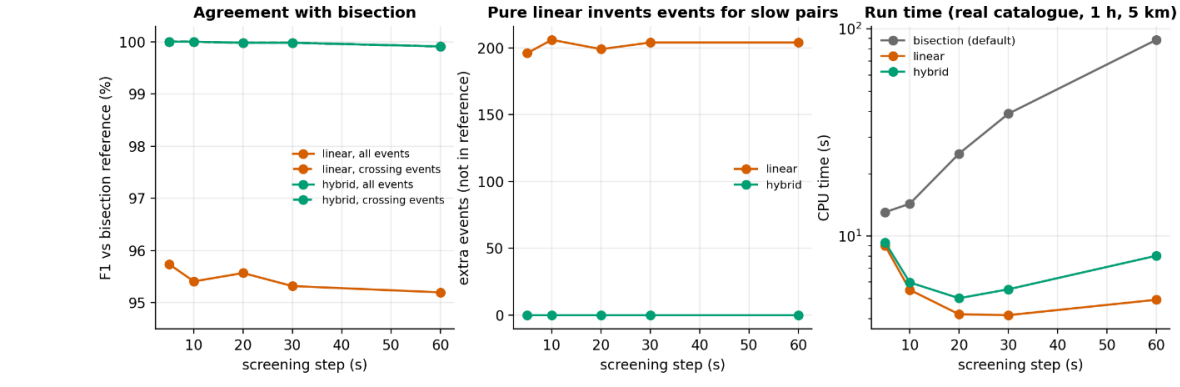


Fig. 24. Linear interpolation against bisection: precision, recall and F1, and extra events by relative speed.

F. Age-aware screening distance

Satellite element sets grow less accurate with age. The age-aware distance adds g km per day of element age to each object's screening distance. On the real catalogue the median element age is 0.18 days, the 95th percentile 0.71 days and the maximum 2.98 days, so g acts mostly on a small tail of older sets. Even so, g changes the clusters strongly: at g = 0, 0.5, 1, 1.5, 2 and 3 km/day the largest cluster holds 24, 93, 169, 373, 1,354 and 2,502 objects and the events number 2,254, 2,629, 3,046, 3,497, 3,900 and 4,686 (Fig. 25, *results/c8_age_calibration.csv*).

Measured growth. We then measured how the error grows. We downloaded one week (2026-10-16 to 2026-10-23) of element-set history from Space-Track for 296 LEO objects chosen at random (seed 1), giving 75,582 pairs of an older and a newer set of the same object up to 5 days apart. For each pair we propagated the older set with SGP4 to the epoch of the newer set and measured the distance to the newer set's own position (*scripts/calibrate_age_from_history.py*, *results/c8_tle_error_growth.csv*). The median error is 0.26 km for ages up to 0.5 days, 1.6 km for 0.5 to 1 day and 14 km for 4.5 to 5 days. A line through the origin fitted to the medians has a slope of 2.5 km per day, which lies between the g = 2 and g = 3 rows above. The tail is heavy: the 95th percentile is 34 km at 0.5 to 1 day and 1,913 km at 4.5 to 5 days, and the fitted slope for the 95th percentile is 269 km per day. Such a value is not usable as a screening margin, because it would put almost every pair inside the distance. We did not examine which objects form the tail; manoeuvres and strong drag on low orbits are the likely causes, which an element-set age cannot predict.

Three cautions apply. The newer set is treated as the truth, which overstates the error slightly. The sample is 296 objects over one week. And these are errors between two element sets, not against true positions. The default stays g = 0, and all other results in this report use it. If g is switched on, a value near the measured median of 2.5 km per day makes the largest cluster grow from 24 to between 1,354 and 2,502 objects, so the cluster structure at 5 km is sensitive to element-set age.

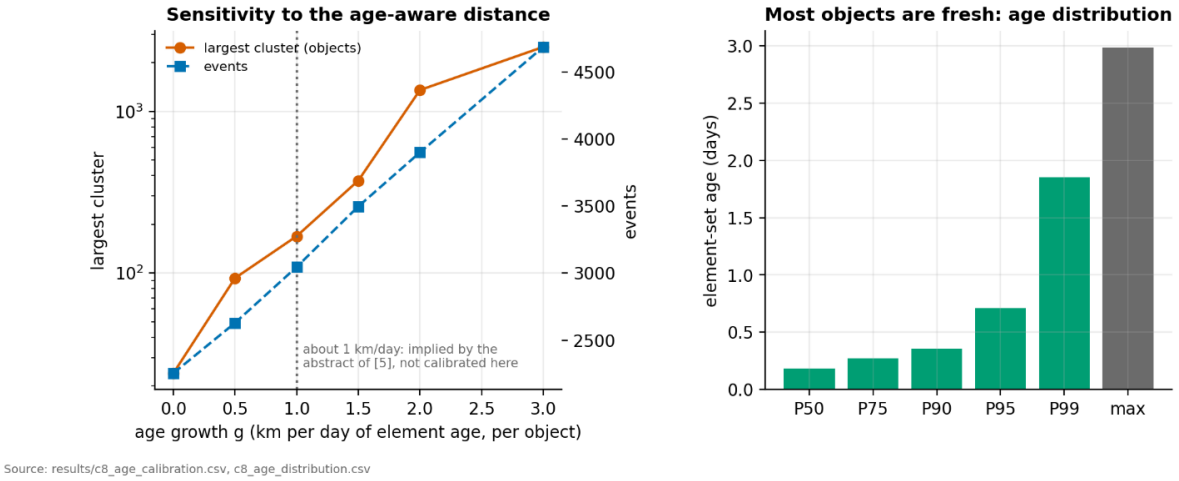


Fig. 25. Age-aware screening distance: largest cluster and events against g, with the element-age distribution.

VIII. LIMITATIONS

The full register is *docs/known_limitations.md* (51 entries). The ones that affect how far the results can be trusted are:

- **Events are SGP4 against SGP4.** The screen is validated against an independent SGP4 implementation, not against reality. We have no covariance, no collision probability and no uncertainty, so a reported conjunction is a geometric closeness in the model, not a risk. The probabilistic side of [2] is outside this work.
- **Scope.** LEO only (175 to 2,000 km). The 805 deep-space objects of the snapshot need SDP4, which is not implemented; the code refuses them. Windows are hours to days, not years.
- **Closest-approach shape.** Slow, co-moving pairs (launch batches, constellations) have no sharp distance minimum. They are within the threshold for much of the window and dominate the list of closest pairs, which is why **Table VI** lists crossing encounters separately.
- **50,000 objects without brute force.** At that size the k-d tree and grid agree with each other (56,780 events) but were not compared with brute force, which would need about 4.5 x 10^{11} pair checks. We claim no speed-up at 50,000.
- **Timing.** CPU seconds on one virtual machine with two logical CPUs. The 10,000-object brute-force point is a single run, and the 20,000, 50,000 and real-catalogue points are single runs. Ratios are the result, absolute times are not.
- **Altitude filter.** It is a heuristic, not a proven bound, and it is slower than brute force in LEO.
- **Hybrid refinement.** The hybrid removes the extra events of linear interpolation but loses one to four events at steps of 20 s and above, and its speed has not been measured. Bisection stays the default.
- **Age-aware distance.** The typical error growth was measured from one week of Space-Track history for 296 objects (median about 2.5 km per day), but the tail is heavy (95th percentile about 269 km per day), so no single g is validated as a safe margin. The newer set is taken as truth, and errors are between element sets, not against true positions (**Section VII-F**).
- **Mitigation model.** Moving an object is assumed to remove all its conjunctions, with no fuel, timing, secondary conjunctions or operator coordination. Only objects in the active-satellite file are treated as movable. At 3 h the exact search was cut off by its node limit.

- **Snapshot.** All cluster statistics come from one catalogue snapshot with a 10 km threshold and windows of up to 6 h. They say nothing about long-term cascade (Kessler) dynamics. The base paper [1] studies 20 years.
- **Tests.** The test programs were run on Linux only. The Windows scripts have never been run, and a known 32-bit long issue on Windows may overflow some counters for very large runs. The three mutants that survived are judged equivalent by inspection, not proven.
- **Literature.** [2] was read at abstract level only, and we make no comparison with its results.

IX. CONCLUSIONS AND FUTURE WORK

We built a complete conjunction-screening pipeline in C99 from hand-written data structures, and used it to compare ways of finding close pairs. With a proven distance bound, a k-d tree and a hashed grid return exactly the events of brute force while examining about 15,000 times fewer pairs. The measured time falls by 10 times at 20,000 synthetic objects and by 6.6 to 6.9 times on the real catalogue of 18,191 objects, because propagation and refinement then dominate. The two indexes are equally good here. An altitude filter does not help in LEO. Closest-approach refinement by bisection does not depend on the step, whereas linear interpolation invents events for slow, co-moving pairs, which a hybrid rule removes at the price of a few missed events at long steps. Conjunctions form one giant cluster at 10 km, and the exact vertex-cover search, with reductions, matches an independent optimum on two of three windows, while greedy is within 1.4 % of it.

Future work. (1) Repeat the age calibration with a longer history and per orbit class, and decide how to treat manoeuvres. (2) Run brute force at 50,000 objects on a machine without our time limit. (3) Add SDP4 and widen the scope beyond LEO. (4) Cache SGP4 states between steps and test a larger step with a tighter bound. (5) Strengthen the lower bound of the exact cover (LP or Nemhauser-Trotter kernel) so that the giant cluster is always solved. (6) Add uncertainty and collision probability, as in [2]. (7) Run and fix the Windows build.

REFERENCES

[1] P. Gómez, F. Gratl, O. Bösing and D. Izzo, "Deterministic Conjunction Tracking in Long-term Space Debris Simulations," 3rd IAA Conference on Space Situational Awareness (ICSSA), Madrid, 2022. arXiv:2203.06957.

[2] M. Mercurio and P. Singla, "A tree-based approach for efficient and accurate conjunction analysis," *CMES: Computer Modeling in Engineering and Sciences*, vol. 111, no. 3, pp. 229-256, 2016. (Read at abstract level only.)

[3] F. R. Hoots and R. L. Roehrich, "Spacetrack Report No. 3: Models for Propagation of NORAD Element Sets," 1980.

[4] D. A. Vallado, P. Crawford, R. Hujsak and T. S. Kelso, "Revisiting Spacetrack Report #3," AIAA 2006-6753, 2006.

[5] C. Levit and W. Marshall, "Improved orbit predictions using two-line elements," *Advances in Space Research*, 2011. arXiv:1002.2277. (Cited from the abstract.)

[6] CelesTrak, "SOCRATES: Satellite Orbital Conjunction Reports Assessing Threatening Encounters in Space." celestrak.org/SOCRATES. (Page checked 2026-10-20.)

APPENDIX A — REPRODUCING THE RESULTS

The code and all result files are in the project folder. From the project root on Linux or macOS:

1. bash *scripts/build.sh* builds the library and the *dlscms* program.
2. bash *scripts/test.sh* runs all tests with and without sanitizers (about 3 minutes).
3. bash *scripts/run_c7.sh* env|scaling|threshold|step|interp|cell|age|scale50k|summary reruns the experiments of **Section VII** (the 50,000-object point is scale50k).
4. bash *scripts/run_c6.sh* reruns the graph and mitigation results; python3 *scripts/verify_cover.py* checks the covers with an independent solver.
5. bash *scripts/export_c9_data.sh* and then python3 *scripts/plot_c9.py* rebuild all figures in *paper/figures/* from results/.
6. bash *scripts/demo.sh* runs the end-to-end demonstration.

No number in this report was typed in by hand from memory; each comes from the files named in the captions. Timings depend on the machine, and the machine used is recorded in *results/environment.txt*. The CelesTrak snapshot of 2026-10-20 is in *data/raw/*. Space-Track element-set history is not included and, under Space-Track's terms, should be downloaded by each user.

APPENDIX B — FIGURE SOURCES

TABLE IX
WHERE EACH FIGURE COMES FROM.

Fig.	Content	Source in results/ or note
1	Catalogue composition	*dataset_stats.csv*
2	Altitude and inclination	*data/raw/*.csv*
3, 6, 7	Pipeline and schematics	drawn, no data
4	Model error against SGP4	*propagation_error.csv*
5	SGP4 port validation	*sgp4_validation.csv*
8	Closest-approach refinement	*c9_pair_curve.csv*
9	Kepler-family agreement with SGP4	*event_agreement.csv*
10	Conjunction graph	*c7_events_thr5.csv*
11	Verification summary	*c8_test_counts.csv*, *c8_coverage.csv*, *c8_mutation.csv*
12, 13, 14	Scaling: time, pair checks, memory	*c7_scaling.csv*, *c7_scaling_50k.csv*, *bench_c5_summary.csv*
15	Structure micro-benchmark	*bench_c1.csv*
16	Grid cell size	*c7_cell.csv*
17	Step size	*c7_step.csv*
18	Altitude filter	*bench_c5_summary.csv*
19, 20	Clusters and timeline	*c7_threshold.csv*, c7_thr**_components.csv*, c7_thr**_timeline.csv*
21	Orbit view at the closest encounter	*c9_positions.csv*, *c9_pair_meta.csv*
22, 23	Mitigation	*c6_summary.csv*, c6_**h_verify.csv*
24	Linear against bisection	*c8_l33_interp.csv*, *c7_interp.csv*
25	Age-aware distance	*c8_age_calibration.csv*, *c8_age_distribution.csv*